

Prüfungssimulation – Beispiel 1

Diese Probeprüfung orientiert sich am offiziellen Themenkatalog der IHK (gültig ab Oktober 2024) und an typischen Aufgaben der vergangenen Jahre. Die Aufgaben sind in deutscher Sprache formuliert und werden unmittelbar darunter auf Persisch erläutert. Zu jeder Teilaufgabe folgt ein Musterlösungs-Pseudocode.

Aufgabe 1 – Lineare und Binäre Suche

Teil a – Lineare Suche

Aufgabe: Schreiben Sie einen Pseudocode für eine lineare Suche, die nach einem ganzzahligen Wert x in einem Array A von Ganzzahlen sucht. Die Funktion soll den Index des gefundenen Elements oder -1 zurückgeben, wenn x nicht in A vorkommt.

Lösung

```
funktion lineareSuche(A: Array von Integer, x: Integer): Integer
    für i = 0 bis A.length - 1
        wenn A[i] = x dann
            Rückgabe i
        ende wenn
    ende für
    Rückgabe -1
endfunktion
```

توضیح سؤال: در این بخش باید الگوریتم جستجوی خطی نوشته شود. این الگوریتم عنصر مورد نظر را با پیمایش تکتک عناصر آرایه پیدا می‌کند و اگر پیدا نشد مقدار منفی برمی‌گرداند.

توضیح پاسخ: کد فوق از یک حلقه برای مرور همه عناصر آرایه استفاده می‌کند و هرگاه مقداری برابر با x یافت شود، اندیس آن بازگردانده می‌شود. در غیر این صورت مقدار -1 (معنای «یافت نشد») بازگردانده می‌شود.

Teil b – Binäre Suche

Aufgabe: Schreiben Sie einen Pseudocode für die binäre Suche. Es wird vorausgesetzt, dass das Array A aufsteigend sortiert ist. Die Funktion soll den Index des gesuchten Wertes x oder -1 zurückgeben.

Lösung

```

funktion binaereSuche(A: Array von Integer, x: Integer): Integer
    links = 0
    rechts = A.length - 1
    während links ≤ rechts
        mitte = (links + rechts) div 2
        wenn A[mitte] = x dann
            Rückgabe mitte
        sonst wenn A[mitte] > x dann
            rechts = mitte - 1
        sonst
            links = mitte + 1
        ende wenn
    ende während
    Rückgabe -1
endfunktion

```

توضیح سؤال: در این بخش باید الگوریتم جستجوی دودویی نوشته شود. این الگوریتم روی آرایه‌ای که به صورت صعودی مرتب شده است کار می‌کند و با تقسیم محدوده جستجو به دو نیم، عدد مورد نظر را سریع‌تر پیدا می‌کند.

توضیح پاسخ: در ابتدای کد، دو متغیر `links` و `rechts` برای محدوده جستجو تنظیم می‌شود. در هر تکرار، موقعیت میانی محاسبه و با مقدار مورد نظر مقایسه می‌شود. اگر مقدار میانی بزرگتر باشد، نیمه چپ بررسی می‌شود و اگر کوچکتر باشد، نیمه راست. در صورت یافتن مقدار، اندیس بازگردانده می‌شود؛ در غیر این صورت مقدار -1 برگردانده می‌شود.

Teil c – Vergleich der Suchmethoden

Aufgabe: Beschreiben Sie die Vor- und Nachteile der linearen und der binären Suche und geben Sie deren Zeitkomplexität an.

Lösung

Die lineare Suche benötigt im schlimmsten Fall n Vergleiche (Zeitkomplexität $O(n)$), ist dafür aber für ungeordnete Daten geeignet und sehr leicht zu implementieren. Die binäre Suche hat eine Zeitkomplexität von $O(\log n)$, funktioniert jedoch nur bei sortierten Daten und erfordert zusätzliches Management des Suchbereichs.

توضیح سؤال: در این بخش باید مزایا و معایب هر دو الگوریتم و پیچیدگی زمانی آن‌ها بیان شود.

توضیح پاسخ: جستجوی خطی برای آرایه‌های نامرتب هم قابل استفاده است ولی در بدترین حالت باید همه عناصر را بررسی کند ($O(n)$). جستجوی دودویی بسیار سریع‌تر است ($O(\log n)$) اما فقط برای آرایه‌های مرتب کار می‌کند و نیاز به نگهداری محدوده جستجو دارد.

Aufgabe 2 – Sortieralgorithmen

Teil a – Bubble Sort

Aufgabe: Schreiben Sie einen Pseudocode für das Bubble-Sort-Verfahren, um ein Array A von Ganzzahlen aufsteigend zu sortieren.

Lösung

```
funktion bubbleSort(A: Array von Integer): void
    n = A.length
    für i = 0 bis n - 2
        für j = 0 bis n - 2 - i
            wenn A[j] > A[j + 1] dann
                // Elemente vertauschen
                temp = A[j]
                A[j] = A[j + 1]
                A[j + 1] = temp
            ende wenn
        ende für
    ende für
endfunktion
```

توضیح سوال: در این بخش باید الگوریتم مرتب‌سازی حبابی نوشته شود که عناصر را با مقایسه زوج‌های مجاور و جابجایی آن‌ها مرتب می‌کند.

توضیح پاسخ: حلقه بیرونی کنترل می‌کند که چند بار کل آرایه پیمایش شود. حلقه داخلی عناصر مجاور را مقایسه می‌کند و در صورت ناهمخوانی (اگر عنصر جلویی بزرگتر باشد) آن‌ها را جابجا می‌کند. پس از هر دور کامل، بزرگترین عنصر بخش بررسی‌شده به انتهای آرایه منتقل می‌شود.

Teil b – Insertion Sort

Aufgabe: Schreiben Sie einen Pseudocode für das Insertion-Sort-Verfahren.

Lösung

```
funktion insertionSort(A: Array von Integer): void
    n = A.length
    für i = 1 bis n - 1
        key = A[i]
        j = i - 1
        während j ≥ 0 und A[j] > key
            A[j + 1] = A[j]
            j = j - 1
        ende während
        A[j + 1] = key
    ende für
endfunktion
```

توضیح سؤال: مرتب‌سازی درجی (Insertion Sort) در هر مرحله یک عنصر از بخش نامرتب را در بخش مرتب در جای صحیح خود قرار می‌دهد.

توضیح پاسخ: در این الگوریتم، حلقه بیرونی عنصر جاری را به عنوان key می‌گیرد. حلقه داخلی عناصر بخش مرتب را که بزرگتر از key هستند یک خانه به جلو می‌برد تا جای خالی ایجاد شود. سپس key در جای خالی قرار می‌گیرد. این کار تکرار می‌شود تا کل آرایه مرتب شود.

Teil c – Vergleich

Aufgabe: Vergleichen Sie die Zeitkomplexität von Bubble Sort und Insertion Sort und nennen Sie typische Einsatzgebiete.

Lösung

Beide Algorithmen haben im schlimmsten Fall eine Zeitkomplexität von $O(n^2)$. Bubble Sort führt immer die gleiche Anzahl an Vergleichen und Vertauschungen durch; Insertion Sort hingegen ist bei fast sortierten Daten deutlich schneller, da Elemente nur verschoben werden, wenn sie wirklich nicht in die Reihenfolge passen. Deshalb eignet sich Insertion Sort besser für Daten, die bereits teilweise sortiert sind.

توضیح سؤال: باید سرعت و کاربرد دو الگوریتم مرتب‌سازی مقایسه شود.

توضیح پاسخ: هر دو الگوریتم در بدترین حالت دارای پیچیدگی $O(n^2)$ هستند. Bubble Sort همیشه زوج‌های مجاور را بررسی می‌کند، در حالی که Insertion Sort در مواردی که آرایه تقریباً مرتب است عملکرد بهتری دارد، زیرا تعداد جابه‌جایی کمتر خواهد بود.

Aufgabe 3 – Datenanalyse und Grundlegende Algorithmen

Teil a – Mittelwert berechnen

Aufgabe: Schreiben Sie einen Pseudocode, der den Mittelwert aller Werte in einem Array werte vom Typ Integer berechnet. Das Ergebnis soll als Fließkommazahl zurückgegeben werden.

Lösung

```
funktion berechneMittelwert(werte: Array von Integer): Double
    summe = 0
    für i = 0 bis werte.length - 1
        summe = summe + werte[i]
    ende für
    wenn werte.length = 0 dann
        Rückgabe 0.0
```

```

sonst
    Rückgabe summe / werte.length
ende wenn
endfunktion

```

توضیح سؤال: باید مقدار متوسط یا میانگین عناصر آرایه حساب شود.

توضیح پاسخ: ابتدا مجموع تمام عناصر محاسبه می‌شود و سپس مجموع بر تعداد عناصر تقسیم می‌شود. اگر آرایه خالی باشد، صفر بازگردانده می‌شود تا از تقسیم بر صفر جلوگیری شود.

Teil b – Doppelte Einträge entfernen

Aufgabe: Schreiben Sie einen Pseudocode, der doppelte Werte aus einem ganzzahligen Array A entfernt und ein neues Array ohne Duplikate zurückgibt. Die Reihenfolge der ersten Vorkommen soll erhalten bleiben.

Lösung

```

funktion entferneDuplikate(A: Array von Integer): Array von Integer
    neuesArray = leeres Array
    für i = 0 bis A.length - 1
        vorhanden = falsch
        für j = 0 bis neuesArray.length - 1
            wenn neuesArray[j] = A[i] dann
                vorhanden = wahr
                breche innere Schleife ab
            ende wenn
        ende für
        wenn vorhanden = falsch dann
            füge A[i] an neuesArray an
        ende wenn
    ende für
    Rückgabe neuesArray
endfunktion

```

توضیح سؤال: هدف این بخش حذف مقادیر تکراری از آرایه است.

توضیح پاسخ: یک آرایه جدید ایجاد می‌شود و هر عنصر آرایه اصلی بررسی می‌شود. اگر عنصر در آرایه جدید وجود نداشته باشد، اضافه می‌شود. بدین ترتیب تنها اولین تکرار هر عنصر نگه داشته می‌شود.

Teil c – Zinseszinsberechnung

Aufgabe: Schreiben Sie einen Pseudocode zur Berechnung des Endkapitals K_n nach n Perioden für ein Anfangskapital K_0 und einen jährlichen Zinssatz p (in Prozent) unter Annahme des Zinseszinses. Verwenden Sie die Formel $K_n = K_0 \times (1 + p / 100)^n$.

Lösung

```
funktion berechneZinseszins(K0: Double, p: Double, n: Integer): Double
    faktor = 1 + (p / 100)
    ergebnis = K0
    für i = 1 bis n
        ergebnis = ergebnis × faktor
    ende für
    Rückgabe ergebnis
endfunktion
```

توضیح سؤال: در این بخش باید نحوه محاسبه سود مرکب را پیاده‌سازی کنید.

توضیح پاسخ: ابتدا عامل افزایش (۱ + نرخ بهره) محاسبه می‌شود و سپس در یک حلقه n باری سرمایه اولیه ضرب در این عامل می‌شود تا سرمایه نهایی به دست آید.

Aufgabe 4 – Rekursion und Dateibaum-Kopie

Teil a – Rekursive Dateibaum-Kopie

Aufgabe: Schreiben Sie einen Pseudocode, der alle Dateien und Unterverzeichnisse von einem Quellverzeichnis `quelle` in ein Zielverzeichnis `ziel` rekursiv kopiert.

Lösung

```
prozedur kopiereVerzeichnis(quelle: String, ziel: String)
    liste = listeDerEinträge(quelle) // Dateien und Unterordner
    für jeden eintrag in liste
        wenn eintrag ein Verzeichnis ist dann
            neuerPfad = ziel + "/" + nameVon(eintrag)
            erstelleVerzeichnis(neuerPfad)
            kopiereVerzeichnis(eintrag, neuerPfad) // Rekursiver Aufruf
        sonst
            kopiereDatei(eintrag, ziel + "/" + nameVon(eintrag))
        ende wenn
    ende für
endprozedur
```

توضیح سؤال: باید یک الگوریتم بازگشتی برای کپی کردن تمام محتویات یک پوشه نوشته شود.

توضیح پاسخ: ابتدا لیستی از فایل‌ها و پوشه‌های داخل مسیر منبع تهیه می‌شود. اگر یک آیتم پوشه باشد، یک پوشه جدید در مسیر مقصد ایجاد شده و تابع بازگشتی برای این پوشه فراخوانی می‌شود. اگر آیتم فایل باشد، مستقیماً کپی می‌شود. بدین ترتیب همه زیرپوشه‌ها و فایل‌ها منتقل می‌شوند.

Teil b – Vorteile und Nachteile der Rekursion

Aufgabe: Nennen Sie zwei Vorteile und zwei Nachteile des rekursiven Programmierens.

Lösung

- **Vorteile:** Rekursive Lösungen sind oft kürzer und näher an der mathematischen Definition eines Problems; die Struktur ist elegant und leicht verständlich, wenn das Problem natürlich rekursiv ist (z.B. Bäume).
- **Nachteile:** Rekursion benötigt zusätzlichen Speicher auf dem Aufrufstapel und kann bei großen Eingaben zu Stapelüberlauf führen; iterative Lösungen sind oftmals effizienter und schneller.

توضیح سؤال: درباره مزایا و معایب برنامه‌نویسی بازگشتی پرسش شده است.

توضیح پاسخ: مزیت بازگشت در سادگی و شباهت به تعریف ریاضی است، اما مصرف حافظه پشته و خطر سرریز از معایب آن محسوب می‌شود؛ در بسیاری موارد نسخه تکراری کارا تر است.

Aufgabe 5 – Backup-Planung und Benutzerauthentifizierung

Teil a – Planung regelmäßiger Backups

Aufgabe: Schreiben Sie einen Pseudocode, der für einen Zeitraum von 30 Tagen regelmäßige Backups plant. Gegeben seien das Startdatum `start` (als Ganzzahl, z.B. 1 für den ersten Tag) und das Backup-Intervall `intervall` (in Tagen). Der Pseudocode soll alle Termine ausgeben, an denen ein Backup stattfinden soll.

Lösung

```
prozedur planeBackups(start: Integer, intervall: Integer)
    tag = start
    während tag ≤ 30
        schreibe("Backup am Tag " + tag)
        tag = tag + intervall
    ende während
endprozedur
```

توضیح سؤال: برنامه باید با توجه به فاصله زمانی تعریف شده، تاریخ‌های بکاپ را در طول یک ماه محاسبه کند.

توضیح پاسخ: یک متغیر `tag` با مقدار روز شروع تنظیم می‌شود و در هر تکرار مقدار آن به خروجی نوشته می‌شود، سپس `intervall` به آن افزوده می‌شود تا زمان بعدی محاسبه شود. این روند تا پایان روز ۳۰ ادامه می‌یابد.

Teil b – Benutzerauthentifizierung

Aufgabe: Schreiben Sie einen Pseudocode für eine einfache Benutzerauthentifizierung. Der Benutzer gibt Benutzername und Passwort ein. Diese Daten werden mit den gespeicherten Zugangsdaten verglichen. Im Erfolgsfall wird der Zugang gewährt, andernfalls erfolgt eine Fehlermeldung und ein erneuter Versuch.

Lösung

```
prozedur authentifiziereBenutzer()  
    gespeicherterName = "admin"  
    gespeichertesPasswort = "geheim"  
    versuche = 3  
    während versuche > 0  
        benutzerName = liesEingabe("Benutzername: ")  
        passwort = liesEingabe("Passwort: ")  
        wenn benutzerName = gespeicherterName und passwort = gespeichertesPasswort dann  
            schreibe("Anmeldung erfolgreich")  
            Rückgabe  
        sonst  
            versuche = versuche - 1  
            schreibe("Falsche Daten. Verbleibende Versuche: " + versuche)  
        ende wenn  
    ende während  
    schreibe("Zugang gesperrt")  
endprozedur
```

توضیح سؤال: باید مراحل ورود کاربر شامل دریافت نام کاربری و رمز عبور و کنترل آن‌ها را بنویسید.

توضیح پاسخ: در این الگوریتم ابتدا نام کاربری و رمز عبور صحیح در متغیرهایی ذخیره می‌شود. کاربر تا سه بار فرصت دارد که اطلاعات خود را وارد کند. اگر نام کاربری و رمز عبور درست باشد، پیام موفقیت نمایش داده می‌شود؛ در غیر این صورت از تعداد فرصت‌ها کم شده و در نهایت در صورت اتمام فرصت‌ها، دسترسی مسدود می‌شود.

Aufgabe 6 – Einfache Reguläre Ausdrücke

Teil a – E-Mail-Validierung

Aufgabe: Geben Sie einen regulären Ausdruck an, der eine einfache E-Mail-Adresse in der Form name@domain.tld prüft. Verwenden Sie nur Buchstaben (a–z, A–Z), Ziffern, Punkte, Bindestriche und Unterstriche vor dem @ sowie nach dem @ vor dem Punkt.

Lösung

^[A-Za-z0-9._%+-]+)@([A-Za-z0-9.-]+)\.([A-Za-z]{2,})\$

توضیح سؤال: این بخش از شما می‌خواهد یک الگوی منظم ساده برای بررسی ساختار ایمیل ارائه دهید.

توضیح پاسخ: عبارت منظم ارائه شده ساختار ایمیل‌های ساده را بررسی می‌کند؛ بخش اول نام کاربر شامل حروف، اعداد و علامت‌های مجاز است، سپس علامت @، پس از آن دامنه و در پایان پسوند دامنه با حداقل دو حرف.